JetBot Nano AG

Teleoperation

1 Einführung

In diesem Abschnitt wird der JetBot ferngesteuert. Dazu wird ein Gamepad an den Laptop angeschlossen, auf dem JupyterLab im Browser geöffnet ist. Die Bewegungen der Sticks werden direkt an die Motoren des Roboters weitergegeben. Gleichzeitig wird das Kamerabild live angezeigt, sodass der JetBot auch ohne direkten Sichtkontakt gesteuert werden kann.

Dieses Dokument dient als Begleitmaterial und sollte **parallel** zu den Notebooks geöffnet werden. Diese befinden sich in JupyterLab im Ordner:

`Notebooks/teleoperation/teleoperation.ipynb`

1.1 Gamepad-Controller erstellen

Zu Beginn wird ein `Controller`-Widget erstellt, über das später der Roboter gesteuert wird. Das Widget benötigt den Parameter `index`. Er gibt an, welches Gamepad verwendet werden soll. Das ist wichtig, falls mehrere Controller angeschlossen sind oder ein einzelnes Gamepad vom Rechner als mehrere Geräte erkannt wird.

Den richtigen Index finden

1. Die Seite `http://html5gamepad.com` im Browser öffnen.
2. Einige Tasten auf dem Gamepad drücken.
3. Den `index` des Gamepads merken, das auf die Tasten reagiert.

Anschließend wird dieser Wert im Notebook eingetragen:

```
controller = widgets.Controller(index=1)  # eigenen Index eintragen
display(controller)
```

Auch bei korrektem Index kann zunächst der Text „Connect gamepad and press any button“ erscheinen. Das Gamepad hat sich dann nur noch nicht beim Notebook angemeldet. Nach einem beliebigen Tastendruck erscheint die Anzeige des Controllers.

1.2 Gamepad mit den Motoren verbinden

Bisher ist das Gamepad nur mit dem Notebook verbunden, aber noch nicht mit dem Roboter. Als Erstes werden die beiden vertikalen Achsen der Sticks mit dem linken und rechten Motor verknüpft. Dafür wird die Funktion `dlink` verwendet.

Im Gegensatz zu `link` erlaubt `dlink`, zwischen Quelle (`source`) und Ziel (`target`) eine Umrechnung (`transform`) einzufügen. Das ist hier nötig, da die Achsen des Controllers umgekehrt zur gewohnten Fahrtrichtung sind: Stick nach vorne liefert einen negativen Wert. Eine kleine `lambda`-Funktion dreht das Vorzeichen um.

```
left_link  = traitlets.dlink((controller.axes[1], 'value'),
                             (robot.left_motor, 'value'),
                             transform=lambda x: -x)
right_link = traitlets.dlink((controller.axes[3], 'value'),
                             (robot.right_motor, 'value'),
                             transform=lambda x: -x)
```

Achse	**Funktion**
Linker Stick (vor/zurück)	linker Motor
Rechter Stick (vor/zurück)	rechter Motor

Gesteuert wird also wie bei einem Panzer: Beide Sticks nach vorne fahren geradeaus, ein Stick nach vorne und einer nach hinten drehen den JetBot auf der Stelle.

Wichtig

Sobald diese Zelle ausgeführt ist, bewegt sich der Roboter bei jeder Berührung der Sticks. Den JetBot vorher auf den Boden stellen und nicht auf dem Tisch stehen lassen.

1.3 Live-Kamerabild anzeigen

Damit der Roboter auch ohne Sichtkontakt gesteuert werden kann, wird das Kamerabild im Notebook angezeigt. Dazu wird zunächst ein `Image`-Widget mit einer Größe von 300×300 Pixeln erstellt.

Diese Größe betrifft nur die Darstellung im Browser. Die tatsächliche Auflösung der Kamera und die übertragene Datenmenge bleiben davon unberührt.

Anschließend wird die Kamera mit `Camera.instance()` gestartet. Die Methode erzeugt eine neue Kamera, falls noch keine existiert. Andernfalls wird die bereits vorhandene zurückgegeben.

1.4 Kamera mit dem Bild verbinden

Die Kamera liefert ihre Bilder im Format **BGR8** (Blau, Grün, Rot mit je 8 Bit). Das `Image`-Widget erwartet dagegen komprimierte **JPEG**-Bilder. Deshalb wird beim Verknüpfen die Funktion `bgr8_to_jpeg` als Umrechnung eingefügt:

```
camera_link = traitlets.dlink((camera, 'value'), (image, 'value'),
                              transform=bgr8_to_jpeg)
```

Nun erscheint das Live-Bild im Notebook.

Tipp

Mit einem Rechtsklick auf die Ausgabe einer Zelle und **Create New View for Output** lässt sich das Kamerabild in einem eigenen Fenster anzeigen. So bleibt es sichtbar, während im Notebook weiter nach unten gescrollt wird.

1.5 Not-Aus bei Verbindungsabbruch

Was passiert, wenn die WLAN-Verbindung abbricht? Die Motoren würden mit dem zuletzt empfangenen Wert einfach weiterlaufen, und der JetBot wäre nicht mehr zu stoppen.

Um das zu verhindern, wird ein `Heartbeat` eingerichtet. Er prüft alle 0,5 Sekunden, ob die Verbindung zwischen Browser und Roboter noch besteht. Bricht sie ab, wird automatisch eine Funktion aufgerufen, die

- die Verbindungen von Kamera und Motoren trennt (`unlink`) und
- den Roboter anhält (`robot.stop()`).

Steht die Verbindung wieder, können Kamera und Motoren mit der nachfolgenden Zelle erneut verknüpft werden.

> **Wichtig**
> Diese Zelle nur ausführen, wenn die Verbindungen tatsächlich getrennt wurden. Andernfalls existieren die Verknüpfungen doppelt, und jeder Befehl wird zweimal an den Roboter gesendet.

1.6 Schnappschüsse per Gamepad speichern

Zum Schluss wird eine Taste des Gamepads so belegt, dass sie das aktuelle Kamerabild speichert. Die Bilder landen im Ordner `snapshots/`. Damit sich kein Dateiname wiederholt, wird jeder Name mit `uuid1()` erzeugt. Diese Kennung enthält zusätzlich Datum, Uhrzeit und MAC-Adresse, was später nützlich sein kann.

Gespeichert wird `image.value` statt des Kamerawerts, da das Bild dort bereits im JPEG-Format vorliegt.

Taste	**Funktion**
Linker Stick	linker Motor
Rechter Stick	rechter Motor
Rechte Schultertaste (Taste 5)	Schnappschuss speichern

Neben dem Live-Bild wird jeweils der zuletzt gespeicherte Schnappschuss angezeigt.

> **Hinweis**
> Genau dieses Prinzip, durch den Raum fahren und per Tastendruck Bilder speichern, wird später bei der **Objekterkennung** genutzt, um schnell viele Trainingsbilder zu sammeln.

1.7 Kamera freigeben

Bevor das Notebook geschlossen und der Kernel beendet wird, muss die Kamera ordnungsgemäß freigegeben werden:

```
camera.stop()
```

Wichtig

Die Kamera kann immer nur von einem Notebook gleichzeitig verwendet werden. Wird sie nicht freigegeben, erscheint in anderen Notebooks die Fehlermeldung `Could not initialize camera`. Zusätzlich den Kernel über **Kernel** → **Shut Down Kernel** beenden.

und nun ab an die Challenges!